Distributed Message Queue
Ad Click Event Aggregation
Hotel Reservation System
Ticketing System
Ride Sharing Service
Stock Exchange
Payment System
Digital Wallet
Stock Exchange
The Learning Curriculum

23
Hotel Reservation System

In this chapter, we design a hotel reservation system for a hotel chain such as Marriott International. The design and techniques used in this chapter also applies to other popular booking related services like:

- Airline ticketing
- Car rental reservation system
- Online movie ticket booking system

Step 1 - Understand the Problem and Establish Design Scope

The hotel reservation system is complicated and its components have several dimensions. Instead of designing the entire system, you should ask the interviewer clarification questions to narrow down the scope.

Candidate: What is the scale of the system?
Interviewer: Let's assume we are building a website for a hotel chain that has 5,000 hotels and 1 million rooms in total.

Candidate: Do customers pay when they make reservations or when they arrive at the hotel?
Interviewer: Customers pay in full when they make a reservation.

Candidate: Do customers also have the option to cancel their reservations? Do we need to support other reservation options such as phone calls?
Interviewer: Let's assume people could book a hotel room through the hotel website or app.

Candidate: Can customers cancel their reservations?
Interviewer: Yes.

Candidate: Do we also offer things like meals or amenities?
Interviewer: Yes, let's allow 50% overbooking. In case you do not know, overbooking means the hotel will sell more rooms than they actually have. Hotels do this because some customers typically cancel their reservations.

Candidate: Since we have limited data, I assume the hotel room search is not within the scope. We focus on the following features:

- Customers can book a hotel room.
- Allow for hotel room overbooking.
- Reserve a room.
- Add guests to a reservation.
- Support the cancelling feature.

Interviewer: Sounds good.

Interviewer: One more thing, hotel prices change dynamically. The price of a hotel room depends on how full the hotel is expected to be on a given day. For example, if many rooms are reserved, we assume the price could be different each day.

Candidate: I know this well.

Next, you might want to talk about the most important non-functional requirements.

Non-functional requirements

- Support high concurrency, during peak season or big events, some popular hotels may have a lot of customers trying to book the same room.
- Moderate latency. It's ideal to have a fast response time when a user makes the reservation, but it's acceptable if the system takes a few seconds to process a reservation request.

Back-of-the-envelope estimation

- Assume 5,000 hotels and 1 million rooms in total.
- Assume 70% of the rooms are occupied and the average stay duration is 3 days.
- Estimated daily reservations: 1 million * 0.7 / 3 = 233,333 (rounding up to 250,000)
- Reservations per second = 250,000 / 10^5 seconds = 2.5 (a day = 10 to the power six, the average reservation transaction per second is around two or three.

Next, let's do a rough calculation of the QPS of popular hotels. There are three steps to expand a search query:

1. Geo location detail (geo) (here narrows the region)
2. Date the booking stays (here can confirm the booking availability, and, in turn, number of guests, payment information before booking begins)
3. Reserve a room. Lower QPS on the "hotel" feature to break the room and the room is reserved (transaction).

Let's assume around 10% of users search for their stay and 50% of users stop at the "date" stage before searching the final step. We can also assume that users typically book rooms for 3 days on average. Finally, we can assume that the system is designed to handle 10 times the rough estimate of what the QPS looks like for different steps. We know the final reservation TPS is 3 so we can work backwards using the formula: The QPS of the order confirmation page is 30 and the QPS for the detail page is 300.

Figure 1:QPS estimation

Step 2 - Propose High-Level Design and Get Buy-In

In this section, we'll discuss:

- API design
- Data models
- High-level design

API design

We express the API design for the hotel reservation system. The APIs are based on RESTful conventions.

Note that this chapter focuses on the design of a hotel reservation system. For a complete hotel website, the design needs to provide functionality such as recommendations, ratings, reviews, customer support, and so on. Other APIs for these search features, online impression, personalization, and ranking. They are out of scope for this chapter.

Hotel related APIs

	API	Detail
GET	/v1/hotels/{id}	Get detailed information about a hotel.
POST	/v1/hotels	Add a new hotel. The API is only available to hotel staff.
PUT	/v1/hotels/{id}	Update hotel information. The API is only available to hotel staff.
DELETE	/v1/hotels/{id}	Delete a hotel. The API is only available to hotel staff.

Table 1:Hotel related APIs

Room related APIs

	API	Detail
GET	/v1/rooms/{id}/overall	Get detailed information about a room.
POST	/v1/rooms/{id}/overall	Add a room. The API is only available to hotel staff.
PUT	/v1/rooms/{id}/overall	Update room information. This API is only available to hotel staff.
DELETE	/v1/rooms/{id}/overall	Delete a room. The API is only available to hotel staff.

Table 2:Room related APIs

Reservation related APIs

	API	Detail
GET	/v1/reservations	Get the reservation history of the logged-in user.
GET	/v1/reservations/{id}	Get detailed information about a reservation.
POST	/v1/reservations	Make a new reservation.
DELETE	/v1/reservations/{id}	Cancel a reservation.

Table 3:Reservation related APIs

Making a new reservation is a very important feature. The request parameters of making a new reservation (POST /v1/reservations) could look like this.

```
{
  "startDate": "2021-08-29",
  "endDate": "2021-08-31",
  "hotelID": "12345678",
  "roomID": "87654321",
  "reservationCount": "1000000"
}
```

Please note reservations is used as the idempotency key to prevent double booking. Double booking means multiple reservations are made for the same room on the same day. The details are explained in "Concurrency Issue" in the Deep Dive section.

Data model

Before we delve into the database to use, let's take a close look at the data access patterns. For the hotel reservation system, we need to store the following data:

Query 1: View detailed information about a hotel.

Query 2: Find available types of rooms given a date range.

Query 3: Reserve a room.

Query 4: Look up a reservation or perform other operations.

Based on the business requirements and access patterns, a set of the system is not large but we still need to prepare for traffic surges during big events. With these requirements in mind, we choose a relational database scaled for read operations.

- A relational database works with read-heavy and write-light-heavy popularity workloads. This is because the number of users who use the hotel reservation system is a fraction of magnitude higher than those who make reservations. Hotels, databases and payments are relatively infrequent. Hotel reservations typically happen months in advance.
- A relational database offers ACID properties by its nature. Strict ACID properties are not required for most reservations, ACID properties are important to a reservation system because those properties. It's not an issue with problems such as negative balances, double charges, double reservations, etc. ACID properties make application code a lot simpler and leads the whole system easier to reason about. A relational database supports transactions and isolation.
- A relational database handle the data well. The structure of the business data is very clear and the relationship between different entities (hotel, room, rooms, type, etc) is stable. This kind of data model is easily modelled by a relational database.

Now that we have chosen the relational database as our data store, let's explore the schema design. Figure 2 shows a straightforward schema design which is the most intuitive way for many candidates to think about a hotel reservation system.

Figure 2:Database scheme

Most attributes are self-explanatory and we will only explain the values that fit in the reservation table. The status field can be one of these values (pending, paid, refunded, canceled, rejected) The schema machine is defined in Figure 3.

Figure 3:Reservation states

The schema design has a major issue. This data model works for companies that stores as many or single-use cities selling, all in given when users make reservations. However, this isn't the case for Marriott or any other hotel chain which reserve guest rooms in a given hotel inside a specific room. For instance, a room type can be a standard room, king-size room, queen-size room with two queen beds, etc. Room inventory is managed at the room type level. Based on the data model in Figure 2, we need to update our data model to reflect this new requirement. Our improved schema is the Entity-Relationship diagram in Figure 4.

High-level design

We use the microservices architecture for this hotel reservation system. Over the past few years, microservices architecture has gained tremendous popularity and has been proven to be a scalable and flexible design. Figure 5 shows the high-level design. Today, it's a pure hotel reservation system, but it's flexible enough to accommodate other services such as payment and gift card processing.

Our design is modeled with the microservice architecture and the high-level design diagram is shown in Figure 5.

Figure 4:High-level design

We will briefly go over each component of the system from top to bottom.

- User is a user interface manager that models phone or browser.
- Public API, authorized hotel staff perform administrative operations such as refunding a customer, canceling a reservation, adding a new room type, etc.
- CDN (content delivery network) By better load time, CDN can cache static assets, including JavaScript bundles, images, videos, HTML, etc.
- Load balancer (LB) is a fully managed service that supports cross-binding authentication. LBs. The LB gateway is configured to direct requests to the proper backend. Based on the request path, requests are routed to different services and requests are directed to the hotel service and requests to book a hotel room are routed to the reservation service.
- Hotel service, rate service, and reservation service are stateless, they are accessible through internal software or sidecars. They are usually in charge of providing a RESTful API to the client.
- Hotel DB hosts the provides detailed information on hotels and rooms. Hotel and room data are generally static, so can be easily cached.
- Rate service provides room rates data for different future dates. An interesting fact about the hotel industry is that the price of a room changes dynamically based on how many rooms are booked, how far out the booking is, and so on.
- Reservation service receives reservation requests and manages the hotel rooms. This service also tracks room inventory as rooms are reserved or reservations are canceled.
- Payment service, after a reservation is made, a customer will update the reservation status to "paid" once a payment transaction succeeds, or "rejected" if the transaction fails.
- Hotel Management Service runs asynchronously to authorized hotel staff. Hotel staff and agents will use the following features: view the record of a reservation, cancel a reservation, reserve a room for a customer, or update room information.

For clarity, Figure 5 only shows a subset of interactions between microservices. For example, as shown in Figure 6, there should be an authentication and authorization layer between the public API gateway and the internal services. Also, there should be a service mesh that manages the security of communications between services. Sidecars are usually used to implement such functionality. Instead of hard-coded logic for how services talk to each other, we can delegate cross-cutting the hotel management tasks with most of the other services. When an admin makes changes to Hotel Management Services, the requests are forwarded to the actual services running the data. It handles the changes.

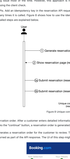

Figure 5:Connections between services

For production systems, inter-service communication often employs a modern API framework such as gRPC. There are many benefits to using such frameworks. To learn more about gRPC in particular, check out (1).

Step 3 - Design Deep Dive

Now we've talked about the high level design, let's go deeper into the following:

- Improved data model
- Concurrency issues
- Scaling the system
- Restricting departments in the microservice architecture

Improved data model

As mentioned in the high-level design, when we reserve a hotel room, we actually reserve a type of room, as opposed to a specific room. Since we are ready to change about the API and schema to accommodate this.

The new API to make a reservation is modified to supply roomTypeID in the request parameters. The API to make a reservation looks like this:

```
POST /v1/reservations
```

Request parameters:

```
{
  "startDate": "2021-08-29",
  "endDate": "2021-08-31",
  "hotelID": "12345678",
  "roomTypeID": "87654321",
  "reservationCount": "1000000"
}
```

The updated schema is shown in Figure 6.

Figure 6:Updated scheme

We'll briefly go over some of the most important tables.

rate: contains information about a room.

room_type_rate: stores price data for a specific room type, for future dates.

inventory: records guest reservation data.

room_type_inventory: stores inventory data about hotel rooms. This table is very important for the reservation system, so let's take a closer look at each column:

- hotel_id: ID of the hotel
- room_type_id: ID of the room type
- date: a single date
- total_inventory: the total number of rooms minus those that are temporarily taken off the inventory. Some rooms might be taken off the inventory for maintenance.
- total_reserved: the total number of rooms booked for the specified hotel, room type, and date.

There are other ways to design this room type inventory table but having one row per date makes managing reservations under a date range and payments very efficient. Figure 7 shows a sample data set, using data in the composite primary key. The rows of the table are populated by querying the inventory status across all future dates within 2 years. We have a scheduled daily job that pre-populates inventory data into the table.

Now that we've finished the schema design, let's do some estimation about the storage volume. As mentioned in the back-of-the-envelope estimation, we assume we are building a system for 1 million rooms in total. Assume the hotel chain has 250,000 rooms that are booked every day (1 million rooms / 4 rooms on average). Let's also assume we keep inventory data for two years. Assume a single piece of lodging, such as a hotel room, requires 2 bytes of storage for data, we need 2 bytes * 1 million rooms * 365 days per year * 2 years = 1.46 billion bytes or roughly 1.46 GB. This is a very small scale and can easily be supported by a single database server, but we consider fault-tolerant and scalable service multiple regions or availability zones.

Table 4 shows the sample data for "room_type_inventory" table.

hotel_id	room_type_id	date	total_inventory	total_reserved
211	1001	2021-06-01	100	101
211	1001	2021-06-02	100	80
211	1001	2021-06-03	100	88
211	1001			
211	1002	2021-06-01	100	0
211	1002	2021-06-02	100	10
211	1002	2021-06-03	100	23
211	1003	2021-06-01	200	160
211	1003	2021-06-02	200	192
211	1003	2021-06-03	200	196

Table 4:Sample data for the "room_type_inventory" table

The room_type_inventory table is utilized to check if a customer can reserve a specific type of room or not. The input and output for a reservation system are as follows:

- Input: parameters (hotel ID, id, arrival date, 2021-01-03), room type(s), number(s)of(reservations)
- Output: a boolean value indicating if the reservation is (true) or not (false). Otherwise, it returns false.

Function fitRoom() accomplishes the following two steps:

1. Select rows within a date range

```
SELECT date, total_inventory, total_reserved
FROM room_type_inventory
WHERE hotel_id = :hotelId AND room_type_id IN (:roomTypeIds)
AND date BETWEEN :startDate AND :endDate
```

This query returns data like this:

date	total_inventory	total_reserved
2021-07-01	100	97
2021-07-02	100	96
2021-07-03	100	95

Table 5:Hotel Inventory

2. For each entry, the application checks the condition:

```
(total_reserved + $(roomNumberRequested)) <= (0.90 * total_inventory)
```

If the condition evaluates to false, the reservation can not be fulfilled for that date within the date range.

One of the requirements is to support 10% overbooking. With overbooking, it is easy to implement.

To the extent, the reservation data spans over a fiscal year, it's common for hotels to change the room type and type distribution, what would you do?. There are four strategies:

- Stop only current and future reservation data. Reservation history is not frequently accessed. So they can be archived and cross-join with other tables, if needed.
- Database sharding. The most frequent queries include looking a reservation by a given hotel, looking up reservations by name, or look for reservations within a date range. Hotel ID is a good sharding key. The data can be sharded by hash(hotel_id) % number_of_servers.

Concurrency Issues

The hotel reservation system is built on a double booking. We need to solve two problems: 1. The same guest clicks on the "book" button multiple times. 2. A certain users try to book the same room at the same time.

Let's take a look at the first scenario. As shown in Figure 7, two reservations are made.

Figure 7:Two reservations are made

There are two common approaches to solve the first problem.

- Client side implementation. A distinct guest key can be added to the backend "system clock" and a request is sent. This should prevent the double-clicking issue. Hence, the backend knows the approach is a duplicate request. Client side implementation is inherently bypassing the client check.
- Unique transaction identifier. Alternatively to the reservation UPI request, the UPI call is idempotent if it produces the same result no matter how many times it is called. Figure 8 shows how to use the idempotency key (reservation id) to avoid the double reservation issue. The detailed steps are explained below.

Figure 8:Unique constraint

1. Generate a reservation order After a customer enters detailed information about the reservation (room type, check-in date, check-out date, and extra) the backend creates a reservation order persisted in the database with a status of "pending".
2. The system generates a reservation order for the customer to review. The unique reservation id is generated by a globally unique ID generator and returned as part of the UPI response. The UI of the payment page looks like this:

Figure 9:Confirmation page (Source: (2))

3. Submit reservation 1. The reservation is in "checkout" as part of the request. It is the primary key of the reservation table (Figure 8). Please note that the reservation key doesn't have to be the reservation id. The chosen reservation id because it already exists and works well for our purpose.
4. 5. If we stick the "Complete my booking" button a second time, everything is a submitted. Because reservation_id is the primary key of the reservation table, the second request will be rejected if the first request is already in the database.

Figure 10 illustrates why double reservation can be problem.

Figure 10:Unique constraint violation

Scenario 2: what happens if multiple users book the same type of room at the same time when there is only one room left? Let's consider the scenario as shown in Figure 11.

Figure 11:Scenario 2

Figure 12 Time condition

1. Let's assume the database isolation level is not serializable (S). User 1 and User 2 try to book the same type of room at the same time, but there is only 1 room left. Let's call User 1's execution "transaction 1" and User 2's execution "transaction 2". At this time, there are 100 rooms in the hotel and 99 of them are reserved.
2. Transaction 2 checks if there are enough rooms left by checking `if total_reserved < rooms_is_book()` => `total_inventory`. Since there is 1 room left, it returns true.
3. Transaction 1 checks if there are enough rooms by checking `if total_reserved < rooms_is_book() => total_inventory`. Since there is 1 room left, it also returns true.
4. Transaction 1 reserves the room and updates the inventory: `reserved_room(rooms_is_book())`.
5. Then transaction 2 reserves the room. The `isolation` property in ACID ensures database transactions must complete their tasks independently from other transactions. So data changes made by transaction 1 are not visible to transaction 2 until transaction 1's operations are completed. Therefore, even after User 1 reserves the last room, User 2 still sees 1 room left and thinks it can reserve it.
6. Transaction 2 reserves the room, resulting in 100 reservations for 99 rooms. Now, we have a room that's overbooked.
7. Transaction 1 successfully commits the change.
8. Transaction 2 successfully commits the change.

The solution to this problem generally requires some form of locking mechanism. We explore the following techniques:

- Pessimistic locking
- Optimistic locking
- Database constraints

Before jumping into a fix, let's take a look at the SQL pseudo code utilized to reserve a room. The SQL has two parts:

- Check if rooms are available
- Reserve a room.

```
-- Step 1: Check room inventory
SELECT date, total_inventory, total_reserved
FROM room_type_inventory
WHERE room_type_id = $(roomTypeId) AND hotel_id = $(hotelId)
AND date between $(startDate) and $(endDate)

-- For every entry returned from step 1
-- $(total_reserved + $(numberOfRooms$(date)) < $(URM * total_inventory) {
-- BEGIN
-- ...
-- }
-- ...
-- step 2: reserve rooms
UPDATE room_type_inventory
SET total_reserved = total_reserved + $(numberOfRooms$(date)
WHERE room_type_id = $(roomTypeId)
AND date between $(startDate) and $(endDate)

Cases:
```

Option 1: Pessimistic locking

Pessimistic locking (PL) also called pessimistic concurrency control, prevents simultaneous updates by placing a lock on a record as soon as one user starts to update it. Other users who attempt to update the record must first wait until the first user has released the lock (committed the changes).

For MySQL, the "SELECT ... FOR UPDATE" statement works by locking the rows returned by a selection query. Let's assume a transaction wrapping the two SQL statements. All transactions have to wait for transaction 1 to finish before beginning their transaction. A detailed explanation is shown in Figure 13.

Figure 13 Pessimistic locking

In Figure 13, the "SELECT ... FOR UPDATE" statement of transaction 2 waits for transaction 1 to finish because transaction 1 locks the rows. After transaction 1 finishes, total_reserved becomes 100, which means there is no room for user 2 to book.

Pros

- Prevents applications from updating data that is being — or has been — changed.
- It is easy to implement and it works correctly for remaining updates. Pessimistic locking is useful when data contention is heavy.

Cons

- Deadlocks may occur when multiple resources are locked. Writing deadlock-free application code could be challenging.
- This approach is not scalable. If a transaction is hoisted for too long, other transactions cannot access the resources. This has a significant impact on database performance, especially when transactions are long-lived or involve a lot of entities.

Due to these limitations, we do not recommend pessimistic locking for the reservation system.

Option 2: Optimistic locking

Optimistic locking (OL), also referred to as optimistic concurrency control, allows multiple concurrent users to attempt to update the same resource.

There are two common ways to implement optimistic locking: version number and timestamp. Version number is generally considered to be a better option because the server clock can be inaccurate over time. We explore two optimistic locking works with version number.

Figure 14 shows a successful case and a failure case.

Figure 14 Optimistic locking

1. A row column called "version" is added to the database table.
2. Before a user modifies a database row, the application reads the version number of the row.
3. When the user updates the row, the application increases the version number by 1 and writes it back to the database.
4. Database accepts the change if the version number is greater than the stored version number or rejects the transaction. The transaction aborts if the version has not the user has again from step 1.

Optimistic locking is usually faster than pessimistic locking because we do not lock the database. However, the performance of optimistic locking drops dramatically when concurrency is high.

To understand why, consider the case when many clients try to reserve a hotel room at the same time. Because there is no lim on how many clients can read the available room value at step 1, all of them read the same available room count and the current version number. When different clients make reservations and write back the results to the database, only one of them will succeed, and the rest of the clients receive a version check failure message. These clients have to retry, in the subsequent round of retries, there is only one successful client again, and the rest have to retry. Although the end result is correct, repeated retries cause a very unpleasant user experience.

Pros

- It prevents applications from editing stale data.
- We don't need to hold the database resources actually no locking from the database point of view. It's entirely up to the application to handle the logic with the version number.
- Optimistic locking is generally used when the data contention is low. When conflicts are rare, transactions can complete without the expense of managing locks.

Cons

- Performance is poor when data contention is heavy.

Optimistic locking is a good option for a hotel reservation system since the QPS for reservations is usually not high.

Option 3: Database constraints

This approach is very similar to optimistic locking. We add invariants to rules. In the room_type_inventory table, add the following constraint.

```
CONSTRAINT check_room_count CHECK(total_inventory - total_reserved >= 0)
```

Using the same example as shown in Figure 14, when user 2 tries to reserve a room, total_reserved becomes 101, which violates the total_inventory (100) - total_reserved (101) >= 0 constraint. The transaction is then rolled back.

Figure 14 Database constraint

Pros

- Easy to implement.
- It works well when data contention is minimal.

Cons

- Similar to optimistic locking, when data contention is heavy, it can result in a high volume of failures. Users could see these are rooms available during room availability checks but fail to book the room because someone else has reserved it.
- The database constraints cannot be version controlled easily like application code.
- Not all databases support constraints. It might cause problems when we migrate from one database solution to another.

Since this approach is easy to implement and the data contention for a hotel reservation is usually not high (low QPS), it is another good option for the hotel reservation system.

Scalability

Usually, the load of the hotel reservation system is not high. However, the interviewer might have a follow-up question: "What if the hotel reservation system is used not just for a hotel chain, but for a popular travel site such as booking.com or expedia.com?" In this case, the QPS could be 1,000 times higher.

When the system load is high, we need to understand what might become the bottleneck. All our services are stateless, so they can be easily scaled by adding more servers. Therefore, the database contains all the states and cannot be scaled up by simply adding more databases. Let's explore how to scale the database.

Database sharding

One way to scale the database is to apply database sharding. The idea is to split the data into multiple databases so that each of them only contains a portion of data.

When we shard a database, we need to consider how to distribute the data. As we can see from the data model section, most queries need to filter by hotel. So a natural conclusion is to shard data based on hotel. In Figure 15, the hotel is spread among 3 shards. Assume the QPS is 30,000. After database sharding, each shard handles 30,000 / 3 = 10 QPS, which is within a single MySQL server's load capacity.

Figure 15 Database sharding

Caching

The hotel inventory data has an interesting characteristic: only current and future hotel inventory dates are meaningful because customers can only book rooms in the near future.

So for the caching choice, ideally we want to have a time-to-live (TTL) mechanism to expire old data automatically. Historical data can be queried on a different database. Redis is a good choice because TTL and Least Recently Used (LRU) cache eviction policy help us make optimal use of memory.

If the leading speed and database scalability became issues (for instance, we are designing at booking.com or expedia.com's scale), we can separate the reserve and query functionalities into different services. This decoupling allows us to optimize the query side by using Elastic Search or Druid. The reservation side will still need to be the source of truth for the inventory data. Changes in the inventory database are most instigated by the inventory cache. One thing worth mentioning is that even when there is enough inventory shown in Redis, we still need to recheck the inventory at the database side as a precaution. The database is the source of truth for the inventory data.

Figure 16 Caching

Let's first go over each component in this system:

Reservation service: supports the following inventory management APIs:

- Query the number of available rooms for a given hotel ID, room type, and date range.
- Reserve a room by executing `total_reserved + 1`.
- Update inventory when a user cancels a reservation.

Inventory cache: all inventory management query operations are directed to the inventory cache (Redis) and we need to pre-populate inventory data to the cache. The cache is a key-value store with the following schema:

```
key: hotel_ID:room_type_ID:date(date)
value: the number of available rooms for the given hotel ID, room type ID and date.
```

For a hotel reservation system, the volume of read operations (where room inventory) is an order of magnitude higher than write operations. Most of the read operations are answered by the cache.

Inventory DB: stores inventory data as the source of truth.

New challenges posed by the cache

Adding a cache layer significantly increases the system scalability and throughput, but it also introduces a new challenge: how to maintain data consistency between the database and the cache.

When a user books a room, two operations are executed in the happy path:

1. Query room inventory to find out if there are enough rooms left. This query runs on the inventory cache.
2. Update inventory data. The inventory DB is updated first. The change is then propagated to the cache asynchronously. This asynchronous update can be implemented by a change data capture (CDC) service. For simplicity, we don't go into details about CDC in this discussion. A cron job can be employed using change data capture (CDC) [5]. CDC is a mechanism that tracks data changes from the database and applies the changes to another data system. One common solution is Debezium [6]. It uses a source connector to read changes from a database and applies them to a sink solution such as Redis, Kafka, or other databases.

Because the inventory data is updated on the database first, there is a possibility that the cache does not reflect the latest inventory data. For example, the same key-value pair as shown in Figure 16 is stored in the cache. Let's assume there is no room left at one date.

If you think carefully, you find that the inconsistency between inventory cache and database actually does not matter, as long as the database does the final inventory validation.

Let's take a look at an example. Let's say the cache shows there is still an empty room, but the database says there is not. In this case, the cache is wrong, and we can rely on the database to make the right decision. The cache will be updated when the database sends the inventory change data. The database blocks the cache update as long as there is no room left. In this case, the cache becomes an error, indicating someone else just booked the last room before them. When a user refreshes the website, they probably expect that there is no room left because the database has accurate inventory data but the cache might be slightly stale.

Pros

- Reduced database load. Since most queries are answered by the cache layer, database load is significantly reduced.
- High performance. Read queries are very fast because results are fetched from memory.

Cons

- Maintaining data consistency between the database and cache is hard. We need to think carefully about how this inconsistency affects user experience.

Data consistency among services

In a traditional monolithic architecture [7], a shared relational database is used to ensure data consistency. In our microservice design, we chose a hybrid approach for having Reservation Service handle room reservation and inventory APIs, so that the inventory and reservation database tables are stored in the same relational database. As explained in the "Concurrency Issues" section, this approach ensures data consistency in the ACID properties of the relational database alleviating handle many concurrency issues that arise during the reservation flow.

However, if your microservice is a microservice journey, there might challenge this hybrid approach. In that mind, for a microservice architecture, each microservice has its own database, as shown on the right of Figure 17.

Figure 17 Monolithic vs microservice

This pure design introduces many data consistency issues. Since this is the first time we cover microservices, let's explore how and why it breaks and how we can fix the issues to make the system more robust. In a monolithic architecture, a single function can update two tables in a single transaction. In a microservice architecture, as shown in Figure 18, different operations can be wrapped within a single transaction to ensure ACID properties.

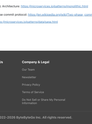

Figure 18 Monolithic architecture

However, in a microservice architecture, each service has its own database. One logically atomic operation can span multiple services. This means we cannot use a single transaction to ensure data consistency. As shown in Figure 19, if the update operation fails in the reservation database, we need to roll back the reserved room count in the inventory database. Generally, there is only one happy path, but many failure cases that could cause data inconsistency.

Figure 19 Microservice architecture

To address the data inconsistency, here is a high-level summary of industry-proven techniques. If you want to read the details, please refer to the reference materials.

- Two-phase commit (2PC) [10]. 2PC is a database protocol used to guarantee atomic transaction commit across multiple nodes, i.e., either all nodes commit or all nodes abort. Because 2PC is a blocking protocol, it is a single-node failure blocks the progress until the failure node recovers. 2PC is not scalable.
- Saga. A saga is a sequence of local transactions. Each transaction updates and publishes a message to trigger the next transaction step. If a step fails, the saga executes compensating transactions to undo the changes that were made by preceding transactions [10]. 2PC works as a single commit to perform ACID transactions while Saga consists of multiple steps and relies on eventual consistency.

It is worth noting that addressing data inconsistency between microservices requires some complicated mechanisms that greatly increase overall complexity. Unless the business is at a very large scale, we recommend using a monolithic architecture. For our design, we decided that if we can work it out so each with the more pragmatic approach of solving reservation and inventory data under the same relational database.

Step 4 - Wrap Up

In this chapter, we proposed a design for a hotel reservation system. We started by gathering requirements and calculating a back-of-the-envelope estimation to ensure the scale is in the high-level design, we proposed the API design, the first draft of the data model, and the system architecture diagram. In the deep dive, we explored alternative database schema designs as we realized transactions should be made at the room type level, as opposed to specific rooms. We discussed race conditions in depth and proposed a few potential solutions:

- pessimistic locking
- optimistic locking
- database constraints

We then discussed different approaches to scale the system, including database sharding and using Redis cache. Lastly, we tackled data consistency issues in microservice architecture and briefly went through a few solutions.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Chapter Summary

Reference Material

[1] Booking.com: https://en.wikipedia.org/wiki/Booking.com

[2] What Are The Benefits of Microservices Architecture?: https://www.geeksforgeeks.org/what-are-the-benefits-of-microservices/

[3] gRPC: https://www.ibm.com/cloud/learn/grpc-a-beginners-guide

[4] Source: Booking.com IDS app

[5] Change data capture: https://en.wikipedia.org/wiki/Change_data_capture

[6] Debezium: https://debezium.io/

[7] Redis site: https://redis.io/topics/lru-cache#approximated-lru-algorithm-benefits-and-consequences

[8] Monolithic Architecture: https://microservices.io/patterns/monolithic.html

[9] Two-phase commit protocol: https://en.wikipedia.org/wiki/Two-phase_commit_protocol

[10] Saga: https://microservices.io/patterns/data/saga.html

Mark as Complete

Partner With Us
Be an Affiliate
Become a Partner

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hello@bytebytego.com
Report A Bug

Resources
YouTube
Visual Bite Codes
Find Jobs
Frequently Asked Questions

Copyright ©2023-2026 Bytebytego Inc. All rights reserved.